

ConnectWise Project – ConnectWise PSA ↔ osTicket Integration

Create a **new project** named:

ConnectWise

This project will be a production-ready integration between **ConnectWise PSA** and **osTicket**.

Project Goal

Develop a modular, scalable, secure, and production-ready integration that synchronizes data between ConnectWise PSA and osTicket.

The architecture should support future integrations with additional PSA platforms such as Autotask PSA, HaloPSA, SuperOps, and Syncro without major code changes.

The project should follow enterprise software architecture and best practices.

Technology Stack

Use:

- PHP 8.3+
- Composer
- MySQL
- Bootstrap 5
- JavaScript
- HTML5
- CSS3
- REST API
- JSON
- PSR-4 Autoloading
- MVC Architecture
- SOLID Principles
- Repository Pattern
- Service Layer
- Dependency Injection where appropriate

Do not use outdated PHP code.

Folder Structure

Create a clean project structure.

```
ConnectWise/  
  
app/  
  Controllers/  
  Models/  
  Services/  
    ConnectWise/  
    OSTicket/  
    Sync/  
  Repositories/  
  Helpers/  
  Middleware/  
  Config/  
  
public/  
  
routes/  
  
storage/  
  logs/  
  cache/  
  
database/  
  
vendor/  
  
resources/  
  views/  
  assets/  
  
tests/  
  
docs/  
  
.env.example  
composer.json  
README.md
```

Configuration

Create an admin configuration page.

Settings should include:

ConnectWise

- Company ID
- Site URL
- Public Key
- Private Key
- Client ID
- API Version
- Board
- Default Status
- Default Priority

osTicket

- API URL
- API Key
- Database Settings
- Department
- Default SLA
- Ticket Source

Allow settings to be updated without editing files.

Store configuration securely.

Authentication

Implement ConnectWise REST API authentication.

Support:

- Company ID
- Public Key
- Private Key
- Client ID

Handle:

- Expired authentication
- Invalid credentials
- Rate limiting
- Retries
- Error logging

ConnectWise API

Create a reusable API client.

Support:

- GET
- POST
- PUT
- PATCH
- DELETE

Automatic pagination

Automatic retries

Timeout handling

Logging

JSON serialization

Exception handling

osTicket API

Create reusable service classes.

Support:

- Create Ticket
- Update Ticket
- Close Ticket
- Add Notes

- Add Attachments
 - Update Priority
 - Update Status
 - Update Agent
-

Ticket Synchronization

Implement two-way synchronization.

Synchronize:

- Ticket
- Status
- Priority
- Company
- Contact
- Owner
- Technician
- Notes
- Attachments
- Category
- Queue
- Department
- Custom Fields

Prevent duplicate ticket creation.

Maintain mapping between systems.

Example:

ConnectWise Ticket #4512

↓

osTicket ID 1256

Store mapping table.

Database

Create migration scripts.

Tables:

sync_ticket_map

sync_logs

sync_queue

sync_history

api_logs

settings

companies

contacts

Include indexes and foreign keys.

Sync Engine

Create a synchronization engine.

Features:

- Initial Sync
 - Incremental Sync
 - Scheduled Sync
 - Manual Sync
 - Retry Queue
 - Failed Queue
 - Conflict Detection
-

Webhooks

Support ConnectWise Webhooks.

Automatically synchronize:

New Ticket

Updated Ticket

Closed Ticket

Note Added

Attachment Added

Priority Changed

Status Changed

Owner Changed

Scheduler

Implement cron jobs.

Examples:

Every Minute

Process Queue

Retry Failed Jobs

Cleanup Logs

Health Check

Logging

Create logging module.

Log:

API Calls

Errors

Sync Events

Authentication

Performance

Request IDs

Store logs in database and files.

Dashboard

Create a Bootstrap dashboard.

Widgets:

Total Tickets

Successful Sync

Failed Sync

Pending Queue

API Health

Recent Activity

System Status

Charts

Last Synchronization

Queue Management

Build queue processing.

States:

Pending

Processing

Completed

Failed

Retry

Dead Letter Queue

Error Handling

Handle:

Network Errors

Authentication Errors

API Errors

Validation Errors

Duplicate Records

Database Errors

Unexpected Exceptions

Conflict Resolution

If ticket changes in both systems:

Detect conflict.

Show differences.

Allow:

Keep ConnectWise

Keep osTicket

Merge

Skip

Mapping

Allow mapping between systems.

Priority

Status

Department

Queue

Ticket Types

Companies

Contacts

Users

Technicians

Security

Implement:

CSRF Protection

Prepared Statements

Input Validation

Output Escaping

XSS Protection

SQL Injection Protection

Rate Limiting

Audit Logs

Secure Session Management

Environment Variables

Performance

Implement:

Caching

Lazy Loading

Pagination

Batch Processing

Queue Processing

Optimized SQL Queries

Connection Reuse

API Documentation

Generate documentation.

Include:

Authentication

Endpoints

Request Examples

Response Examples

Error Codes

Webhook Payloads

Unit Testing

Write PHPUnit tests.

Cover:

API Client

Sync Engine

Repositories

Controllers

Services

Utilities

Installer

Create a setup wizard.

Steps:

Environment Check

Database Connection

Migration

Configuration

API Validation

Admin Account

Finish Installation

Admin Interface

Create pages for:

Dashboard

Settings

Sync Queue

Logs

Users

Mappings

Companies

Contacts

Tickets

System Health

API Tester

Cron Status

Future Expansion

Design the project so new PSA connectors can be added easily.

Example:

PSAInterface

ConnectWiseConnector

AutotaskConnector

HaloConnector

SyncroConnector

SuperOpsConnector

Documentation

Generate:

README.md

Installation Guide

API Guide

Architecture Guide

Developer Guide

Deployment Guide

Troubleshooting Guide

Code Quality

Use:

PSR Standards

Type Hinting

PHPDoc

Meaningful Variable Names

Meaningful Class Names

Strict Types

No duplicated code

Reusable components

Deliverables

Build the project incrementally.

For each completed module:

1. Explain what was built.
2. Show the folder structure.
3. Generate the complete source code.
4. Explain how to test it.
5. Wait for approval before moving to the next module.

Do not skip any implementation details or leave placeholders. Produce production-ready code suitable for deployment.